



University of
Southern Denmark

LARGE-SCALE DRONE PERCEPTION

DRONE MASTER

2. SEMESTER

Visual Odometry Miniproject

made by:

Adam Kolodziej adkol23@student.sdu.dk

Lucas R. Jeppesen ljepp19@student.sdu.dk

Oliver Torngaard oltor19@student.sdu.dk

June 1, 2026

Contents

1	Dataset	3
1.1	Logfile information	3
1.2	Saving specific frames from the video	4
2	Map initialization	5
2.1	Choosing feature detection	5
2.2	Estimate essential matrix	5
2.3	Summary statistics	6
2.4	Relative motion between frames	7
3	3D map initialization	7
3.1	3d point estimation	7
3.2	3d map construction	7
3.3	Reprojection error	8
3.4	Bundle adjustment	9
4	Estimate camera trajectory	9
4.1	Match images to map	9
4.2	Estimate camera positions	9
4.3	Add observations	10
4.4	Camera trajectory visualization	10
5	results	11

List of Figures

1	Plot of the whole flight	3
2	Plot of the flight when recording was enabled (a little bit more than the video we got) .	4
3	feature matching	6
4		6
5	3d map with features and drone postions	8
6	total observations, points, cameras and reprojection error between 2 frames	9
7		11

1 Dataset

1.1 Logfile information

We analyzed the logfile. It is typical file which is generated during every flight. It consists of a lot of data about drones parameters. We can find some obvious data like speed and if the recording is enabled but also data about SD card or even GPS coordinates of the device with DJI Fly app and that it was iOS. What was particularly interesting for us were GPS coordinates of the UAV and the altitude. The data was converted to UTM format using "utm" library created by Tobias Bieniek. Next to be able to plot flight in 3D we used "mplot3d" from "mpl_toolkits". We plotted flight twice. The first one shows the whole flight path and the second one shows only parts when the recording was on. The video we got was recorded on the higher altitude path. On github scripts called: `9_1_1_extractGPS_plot.py`

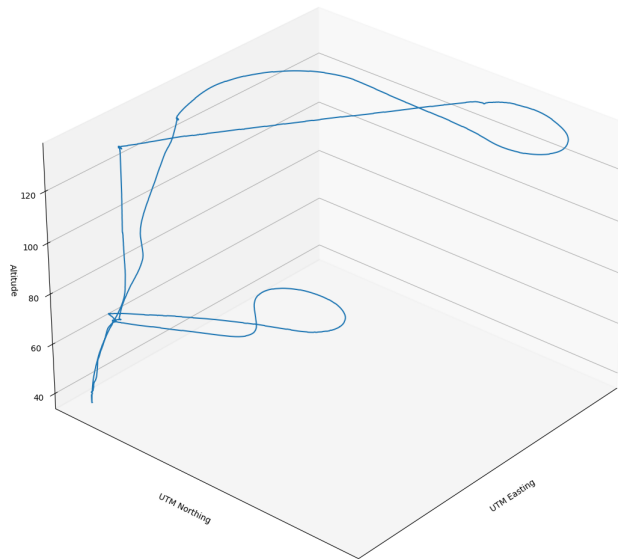


Figure 1: Plot of the whole flight

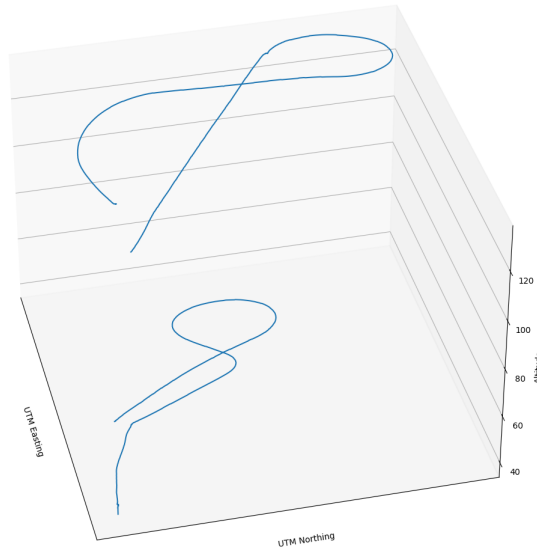


Figure 2: Plot of the flight when recording was enabled (a little bit more than the video we got)

1.2 Saving specific frames from the video

To be able to work on the data we extracted frames from the recorded video. We skipped first 1200 frames and saved only every 25th frame. To achieve that we modified code from lecture "Dealing with videos". Frame generator first skips 1200 frames, then it counts modulo 25 from the i parameter so we know which frame needs to be saved.

On github scripts called: `9_1_2_excludeFrames.py`

Saving every 25th frame gave us better recognition of the drone movement. It allowed us to get information from the data. Only having a frames which are different allows us to estimate the camera position. The keypoints need to change their location on the frame. We also get rid of a lot of data so the computing time will be reduced. When discarding the frames we needed to take into consideration how the video was recorded. If we have a lot of rotations, sharp turns, then we should discard less frames (e.g. take every 10 frames). If the movement is slow, then maybe it would be fine to take e.g. every 100 frames.

During approximately the first 40 seconds of the video the drone was basically stationary. 40 seconds is about 1200 frames. It means that we can't get any useful data from that part - the keypoints are in the same position. After that period drone started moving forward which allows to estimate the

distance between the camera and the keypoints.

2 Map initialization

This section will describe the map initialization. For this project the given code base was used and built upon. This means that the code for feature detection, matching features and creating essential matrices as well as estimating the relative motion between frames, was already in the code base.

2.1 Choosing feature detection

For this project between SIFT and ORB feature detection, it was chosen that the ORB feature detector would be used. The reasoning for this was that the Orb feature detector was a lot faster than SIFT and was able to detect more features than SIFT, while still being able to have the same percentage of matching key points.

Orb is a mix of two functions called FAST and BRIEF, where FAST is used to detect if surrounding pixels are brighter or darker than the center pixel. This method is extremely fast and efficient at detecting features like corners, but is unable to detect the orientation of said features, which is why BRIEF is also used. BRIEF is used to find the orientation of key points and make a pattern of pixels, with can be tested on to find the orientation of a feature.

2.2 Estimate essential matrix

an essential matrix is calculated to get an estimate of the camera movement since the last photo. to get the essential matrix, the ORB feature detector is used on both photos to get the features, after which the matching features a found using Broudforce. in figure 3 the result of feature matching can be seen then the matching features are found, the essential matrix is calculated using RANSAC to estimate the best match. the matrix calculated between the fist 2 frames is

$$\begin{bmatrix} -0.0018 & -0.6376 & -0.2964 \\ 0.6379 & -0.0008 & 0.073 \\ 0.2962 & -0.0754 & -0.001 \end{bmatrix}$$

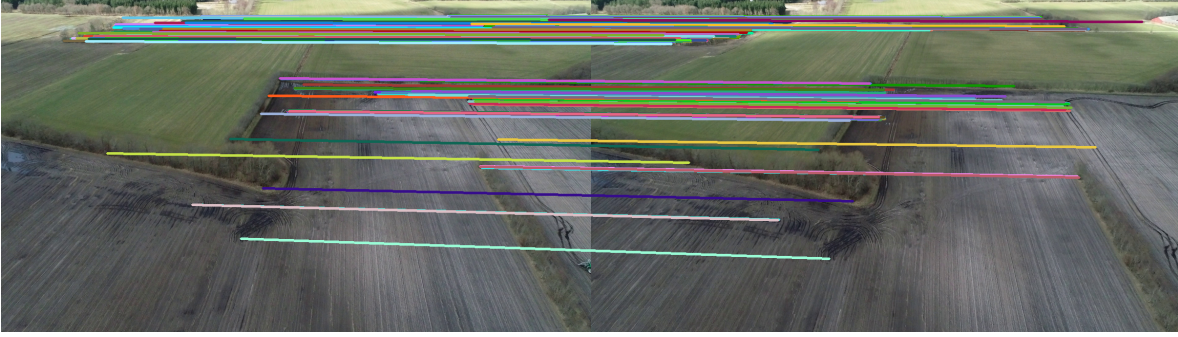
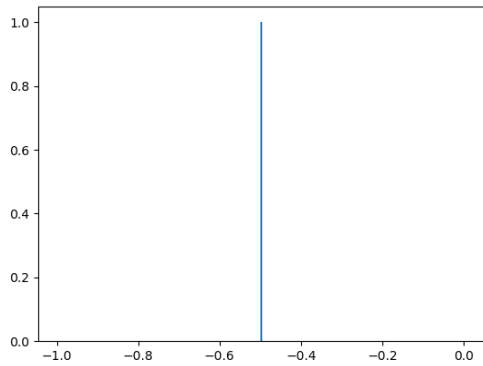


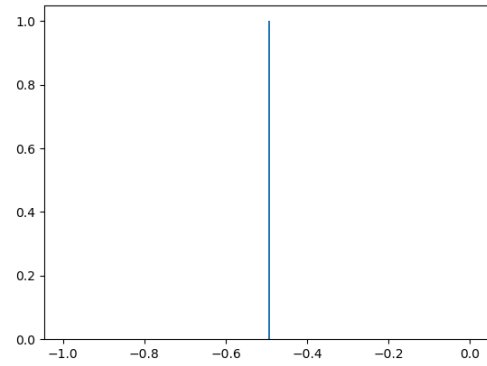
Figure 3: feature matching

2.3 Summary statistics

To calculate the epipolar line between two cameras, a new file called `map_init.py` was made. This class uses Orb to detect points and matches them with two frames, to find the visible observations. It then calculates the essential matrix and generates frames from the images. Afterwards it calculates the epipolar lines with the OpenCV function "computeCorrespondEpilines" and calculates the distance of the line. The mean and standard deviation of the to epipolar lines can be seen in table 1.



(a) histogram of image 1



(b) histogram of image 1

Figure 4

	img1	img2
mean	-0.4954641551071753	-0.4955939667902018
std deviation	0.0	0.0

Table 1

2.4 Relative motion between frames

as the essential matrix mathematical is $E = R[T]_x$ we can calculate the rotation and translate between the to frames. as there is more than one solution to the equation, for each solution the position of the points relative to the camera needs to be checked to make sure the they are in front of the camera.

3 3D map initialization

3.1 3d point estimation

to estimate the 3d position of the features, openCV's **triangulatePoints()** is used, which uses the transform to both cameras and a list of points for photo, this gives the point in reference to the first camera

3.2 3d map contruction

a 3D map of all the features and camera positions can be created as the transformed between camera positions are known and feature positions relative to our camera positions. in figure 5 you can see an 3d map

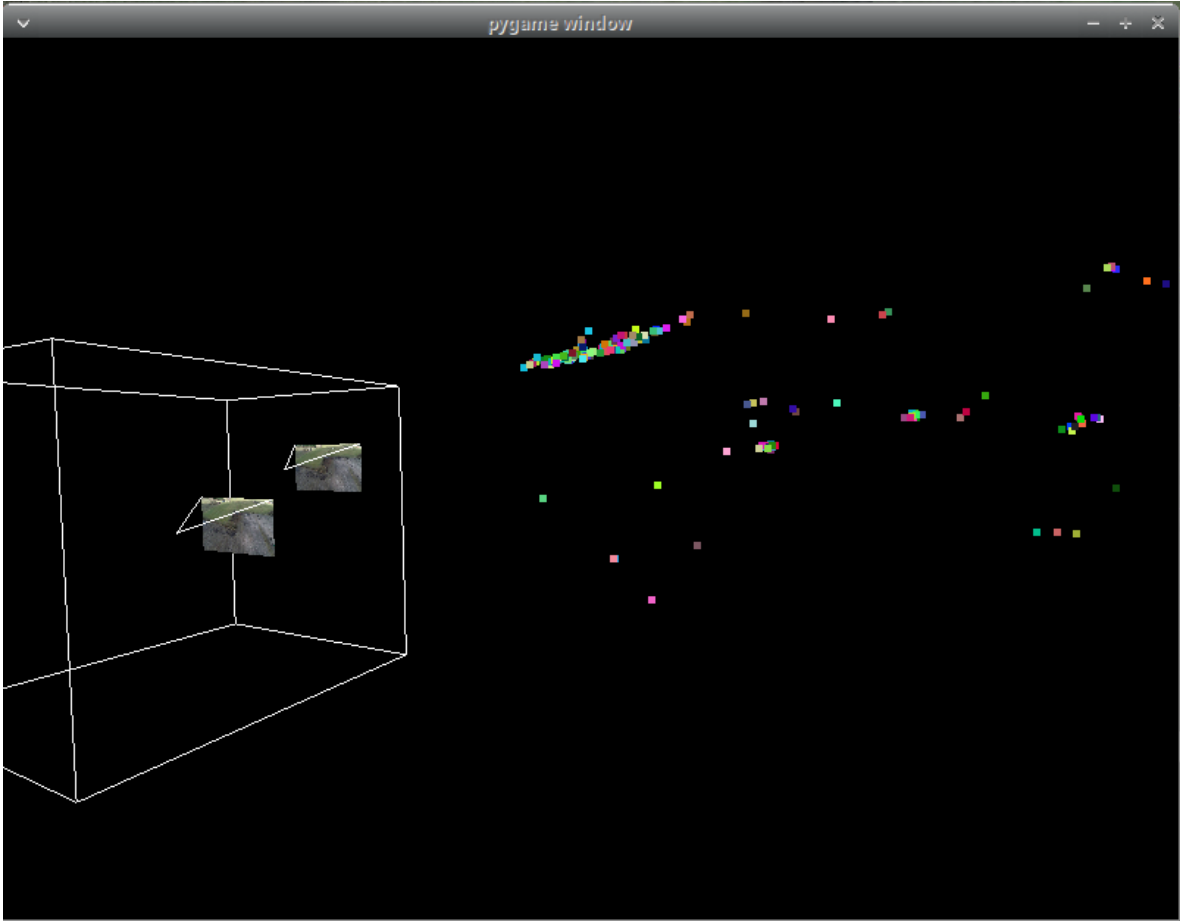


Figure 5: 3d map with features and drone postions

3.3 Reprojection error

To determine how close the observed image is to the expected image, reprojection error is calculated. It says how accurately a 3D point can be projected back onto the image. The final solution should give minimal reprojection error. It is done by optimizing camera position, camera orientation, and feature point locations.

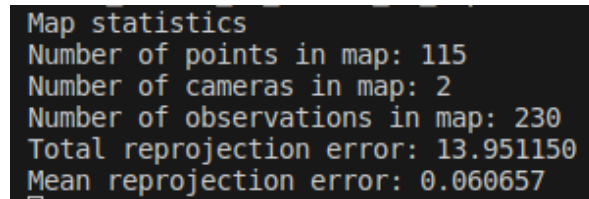
Calculations are done in the "calculate_reprojection_error()" function which gets a parameter threshold. It is used to indicate if the error for a specific observation is high (threshold determines what means "high"). To work fast on data, dictionaries of camera and point objects are created. To calculate the error function iterates through observations. For each observation, the reprojection error is calculated as the squared difference between the reprojected point and the original image coordinates in the observation. Finally, the error is accumulated.

3.4 Bundle adjustment

Bundle adjustment was implemented the g2o python library. It is used to handle optimization problems and is used for this project to handle reprojection error. To use g2o the following elements needs to be addressed.

1. Internal camera properties
2. External camera properties
3. Interest point locations
4. Observations which link cameras and interest points

Where the camera stores pose and id of the cameras, points store the position and id of all tracked points and the observations stores camera and position id, as well as the image coordinates. For this project the function "Optimize_map()" was used to calculate the reprojection error, and the points, cameras and observations between frame 000075 and 000050, as well as the reprojection error are showed in figure 6.

A terminal window with a dark background and light-colored text. It displays the following statistics:

```
Map statistics
Number of points in map: 115
Number of cameras in map: 2
Number of observations in map: 230
Total reprojection error: 13.951150
Mean reprojection error: 0.060657
```

Figure 6: total observations, points, cameras and reprojection error between 2 frames

4 Estimate camera trajectory

4.1 Match images to map

For adding more images to the map, the features the image is compared to the features of the previous image as explained in section 2.2. After matching with the preview image, features are matched with the map by checking if the features exist in the map.

4.2 Estimate camera postions

Estimation of the camera position in the map is done using Perspective-n-point and Ransac to help with outliers, this is done using the OpenCV method `solvePnPRansac()`, the function takes the array of matching features positions in map and image as well as a camera matrix, after which it

returns if a valid camera position was found as well as the rotation and translate of the camera in the map, the rotation and translation of the camera on the 3. photo is

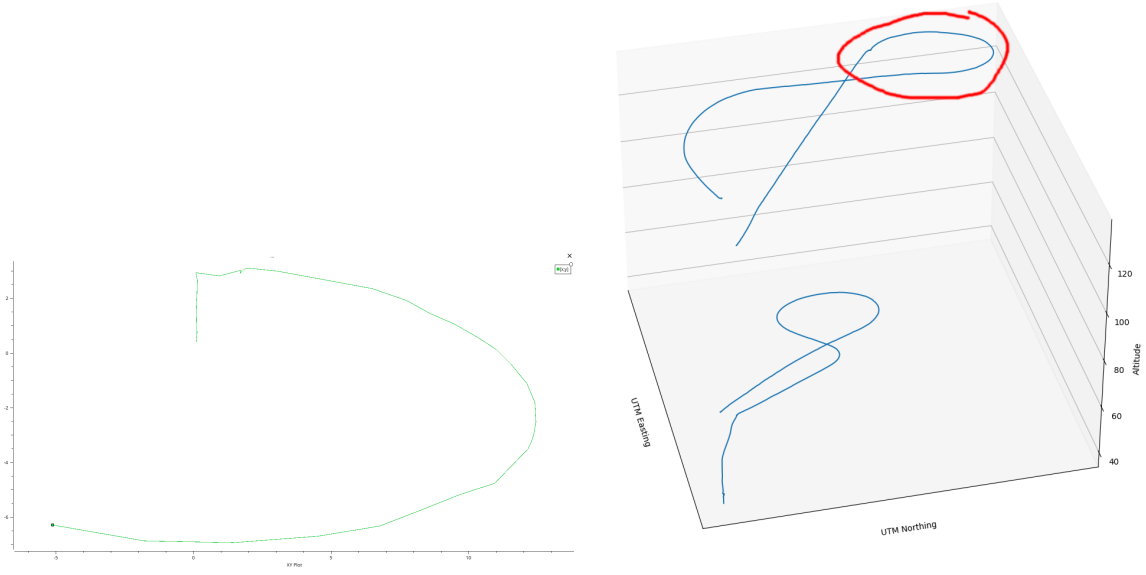
$$R = \begin{bmatrix} 1. & -0.0005 & -0.0045 \\ 0.0005 & 1. & 0.0027 \\ 0.0045 & -0.0027 & 1. \end{bmatrix} T = \begin{bmatrix} 0.1318 \\ 0.7174 \\ -1.7231 \end{bmatrix}$$

4.3 Add observations

To add observations to the map, the estimated camera positions and point estimations are used, as in section 3.4. When the observation has been made it is added by the function "add_point_observation_to_map" to the map, which functions by firstly checking if the point has already been found in another observation, and if it has another observation of said point is added. If the point has not been seen before, the point will be added as a new match to the map, so when future observations find said point, extra observations can be marked down. An example of a number of points found on a map can be found in figure 6.

4.4 Camera trajectory visualization

the camera trajectory can be seen in figure 7a, because of an unknown error in the code the entire path can not be seen, it stops being able to estimate the camera position after around 33 images. the area of the flight path that seems to match the most is marked with a red circle in figure 7b



(a) path of flight generated from photos

(b) flight path with marked area for visual slam math

Figure 7

5 results

feature detection and matching works well using ORB when using every 25th frame, the 3d map of the features and observations looks good until around frame 33 when new frames stop getting matches with the map. Using the features from the ORB feature detector, the essential matrix was computed and used to create matches between frames. With the observations found on the map, it was possible to calculate the epipolar lines between each camera, which showed no deviation. It was possible to calculate the camera trajectory by matching images, estimating camera positions and creating observations of features. The found trajectory was compared to the logged trajectory and showed that they were very similar.

the code can be found here [github](#)